DUTCH POINT NEGOMBO • SOFTWARE ENGINEERING IMPLEMENTATION

Core Business Logic & Algorithmic Specification

System Architecture, Real-Time Concurrency, Dynamic Billing & Viva Defense Guide

Author: *Pasindu Nirmal Iddamalgoda (Student ID: E269559) • Lead Developer*

1. Module 01: Double-Booking Prevention & Concurrency Guard

Double-booking is the most critical operational vulnerability in hotel management systems. It occurs when two separate guests successfully reserve the same physical suite for overlapping dates. The Dutch Point Negombo system eliminates this using temporal interval validation at the database query tier.

CORE HIGHLIGHT: The Double-Booking Rule in Simple English

Two reservations CLASH if and only if BOTH conditions are true simultaneously:
1. The existing booking starts **BEFORE** the new booking ends (ExistingCheckIn < NewCheckOut)
2. The existing booking ends **AFTER** the new booking starts (ExistingCheckOut > NewCheckIn)
• If BOTH are TRUE → **CLASH ■ (Blocked with HTTP 409 Conflict)**
• If EITHER is FALSE → **SAFE ■ (New reservation approved)**

Scenario Analysis (Room #201: Booked Oct 10 to Oct 15)

Scenario	New Dates	CheckIn < End?	CheckOut > Start?	System Decision
Inside Stay	Oct 12 – 14	10 < 14 (True)	15 > 12 (True)	**BLOCKED (409)**
Overlap Start	Oct 08 – 12	10 < 12 (True)	15 > 08 (True)	**BLOCKED (409)**
Overlap End	Oct 13 – 18	10 < 18 (True)	15 > 13 (True)	**BLOCKED (409)**
Full Enclosure	Oct 08 – 20	10 < 20 (True)	15 > 08 (True)	**BLOCKED (409)**
Adjacent Prior	Oct 05 – 10	10 < 10 (False)	15 > 05 (True)	**SAFE (Departs 10th AM)**
Adjacent Next	Oct 15 – 20	10 < 20 (True)	15 > 15 (False)	**SAFE (Arrives 15th PM)**

```
// BookingsController.cs (Lines 152-169)
var conflictingBooking = await _context.Bookings
    .Include(b => b.Customer)
    .Where(b => b.RoomId == request.RoomId &&
               (b.BookingStatus == "Confirmed" || b.BookingStatus == "Checked-In") &&
                b.CheckInDate < checkOut &&
                b.CheckOutDate > checkIn)
    .FirstOrDefaultAsync();

if (conflictingBooking != null)
{
    return Conflict(new {
        message = $"DOUBLE BOOKING PREVENTED: Room {room.RoomNumber} already booked.",
        conflictBookingId = conflictingBooking.BookingId
    });
}
```

Listing 1.1: Database Interval Overlap Query & HTTP 409 Conflict Abort

2. Module 02: Dynamic Multi-Variable Availability & Search Engine

A critical software engineering design decision was avoiding static state mutation on the Room entity. Room availability is inherently temporal—a suite occupied today will be vacant next week. Mutating a static 'Status' column causes permanent lockouts. Instead, availability is computed via in-memory projection.

CORE HIGHLIGHT: Dynamic Projection Architecture

- **Step 1:** System identifies all bookings overlapping with the user's specific check-in / check-out window.
- **Step 2:** Extracts the unique set of occupied Room IDs (.Distinct()).
- **Step 3:** Rooms not present in this occupied set are dynamically marked IsAvailable = true.
- **Step 4:** Filters inventory by party size: Capacity ≥ Guests.

```
// RoomsController.cs (Lines 44-64)
var conflictingBookings = await _context.Bookings
    .Where(b => b.BookingStatus == "Confirmed" || b.BookingStatus == "Checked-In")
    .Where(b => b.CheckInDate < cout && b.CheckOutDate > cin)
    .Select(b => b.RoomId).Distinct().ToListAsync();

var results = rooms.Select(r => new {
    r.RoomId, r.RoomNumber, r.RoomType, r.PricePerNight, r.Capacity,
    DisplayStatus = conflictingBookings.Contains(r.RoomId) ? "Occupied" : r.Status,
    IsAvailable = !conflictingBookings.Contains(r.RoomId) && r.Status == "Available"
});
```

Listing 2.1: Dynamic In-Memory Room Projection based on Date Intervals

3. Module 03: Deterministic Financial Billing & LKR Engine

Hospitality billing requires exact deterministic precision. The Dutch Point Negombo system standardizes all transactions into Sri Lankan Rupees (LKR) with 128-bit decimal fixed-point arithmetic.

- **Nights Computation:** int nights = (checkOut - checkIn).Days; if (nights <= 0) nights = 1;
- **Bill Derivation:** decimal totalAmount = nights * room.PricePerNight; (e.g., 2 Nights × LKR 65,000 = LKR 130,000)
- **Headcount Validation:** request.NumberOfGuests ≤ room.Capacity prevents suite over-occupancy violations.
- **Rounding Safety:** PostgreSQL column typed as decimal(18,2) eliminates IEEE 754 binary floating-point drift.

4. Module 04: Front-Desk Real-Time Occupancy Tracker

Front-desk staff require instantaneous operational visibility during peak arrival hours. The Occupancy Tracker features dual operational filters:

- **Single Target Date Mode:** Evaluates CheckInDate ≤ TargetDate AND CheckOutDate > TargetDate to isolate current in-house guests for any chosen day.
- **Range Sync Mode:** Links directly with the guest booking engine to inspect prospective turn-around conflicts.
- **Guest Manifest Projection:** Pulls guest name, verified Sri Lankan contact (+94), email, billing total (LKR), and housekeeping sanitization status.

5. Module 05: Administrative Governance & Audit Controls

Executive oversight is enforced through strict role separation and historical audit integrity:

- **Client-Side Role Removal:** Public guests cannot toggle administrative privileges; admin routes are guarded by JWT authentication.
- **KPI Aggregation:** Live calculation of Total Yield in LKR (sum(TotalAmount)), resort occupancy percentage, and active guest headcounts.
- **Soft Cancellation & Restoration:** Bookings mutate status flags without destroying historical customer audit trails.

6. Viva Voce Defense Cheat-Sheet (Examiner Q&A;)

Examiner Question	Recommended Viva Voce Answer
'How did you prevent double booking?'	*'Sir/Madam, we enforce interval overlap logic at the database query level: a booking clashes if (ExistingCheckIn < NewCheckOut) AND (ExistingCheckOut > NewCheckIn). If found, the API rejects the request with an HTTP 409 Conflict error.'*
'Why not mutate a static Status column on the Room table?'	*'Because room availability is temporal, not permanent. A room occupied today will be vacant next week. Mutating the room table causes permanent lockouts. We dynamically project availability per date range.'*
'What currency does the system use?'	*'The system strictly operates in Sri Lankan Rupees (LKR) using C# decimal(18,2) types to eliminate binary rounding errors.'*
'How do you protect admin operations?'	*'We removed client-side role pickers from the public UI. Administrative operations are isolated behind secure authenticated endpoints.'*